

A Very Fast Symplectic Integrator for Planetary Systems: WHFast512 in x86 Assembly

RISHIT DAGLI¹ AND HANNO REIN^{1, 2, 3, 4}

¹*Dept. of Computer Science, University of Toronto, Toronto, 40 St. George Street, Toronto, ON M5S 2E4, Canada*

²*Dept. of Physical and Environmental Sciences, University of Toronto at Scarborough, Toronto, Ontario, M1C 1A4, Canada*

³*Dept. of Physics, University of Toronto, Toronto, Ontario, M5S 3H4, Canada*

⁴*Dept. of Astronomy and Astrophysics, University of Toronto, Toronto, Ontario, M5S 3H4, Canada*

ABSTRACT

Long-term integrations of planetary systems are used in a variety of astrophysical applications, for example to determine a system’s stability. In this paper, we describe significant speed and accuracy improvements to the **WHFast512** integrator. By completely rewriting **WHFast512** in x86 assembly, we gain fine-grained control over CPU instructions and are able to keep the entire simulation state in CPU registers, only writing to memory when an output is required. We also reduce the number of branches in the code, improve the Kepler solver’s speed and accuracy with a convergence check, switch the Hamiltonian splitting to Jacobi coordinates, implement symplectic correctors, and add support for ejection and collision detection. As a benchmark we run simulations of the Solar System with all eight planets and general relativistic corrections for 5 Gyr. With our improvements, we can integrate this system in under 8.7 hours, more than $8\times$ faster than the standard version of **WHFast**. This makes **WHFast512** by far the fastest N-body integrator for this kind of simulation. We run extensive tests to verify the integrator’s accuracy, with a particular focus on removing any long-term bias, and show that the accuracy is now five orders of magnitude better for typical setups compared to the earlier version of **WHFast512**. The new **WHFast512** integrator is freely available in the **REBOUND** integrator package.

Keywords: N-body simulations (1083) — Celestial mechanics (211) — Computational methods (1965) — Solar system evolution (2293) — Planetary system evolution (2292)

1. INTRODUCTION

Symplectic integrators that make use of the Wisdom-Holman splitting (J. L. Wisdom 1981; J. Wisdom & M. Holman 1991; H. Kinoshita et al. 1991) are widely used to integrate particles in systems where the motion is dominated by one central object such as planetary systems. Many different variants of the Wisdom-Holman (WH) integrator exist (see e.g. J. Laskar & P. Robutel 2001; D. M. Hernandez & W. Dehnen 2017; H. Rein et al. 2019).

In contrast to many other problems in computational astrophysics, small- N simulations are difficult to parallelize efficiently due to their inherently sequential nature. Each timestep itself can be calculated easily, but a single simulation might require more than 10^{12} timesteps which need to be performed one after another because the result of each timestep is required for the

next one⁵. As such, even on modern CPUs and with access to large computer clusters, an individual N-body simulation can take days, weeks, or even months to run.

Over the years, improvements in computer performance, particularly CPU speed, have significantly reduced the runtime of astrophysical simulations. In Fig. 1 we plot the wall-clock time (elapsed real time) required to integrate the Solar System with all eight planets for 5 Gyr, as reported in 13 publications spanning eight decades. To account for differences in timestep and particle number, the reported simulation efficiency has to be normalized. If t_{wall} is the reported wall-clock time for a simulation integrating N planets for a time t_{sim} , then we define $t_{\text{wall, norm}} = t_{\text{wall}} \cdot (5 \text{ Gyr}/t_{\text{sim}}) \cdot (P_{\text{min}}/88 \text{ d}) \cdot (8/N)^2$, where P_{min} is the shortest orbital period of the simulated planets. As indicated by the dashed line, the wall-clock time falls off exponentially, halving on aver-

⁵ P. Saha et al. (1997) use a time parallel method with some success. The method achieves a speed-up of $50\times$, but it comes with a huge cost: $1000\times$ the computational resources.

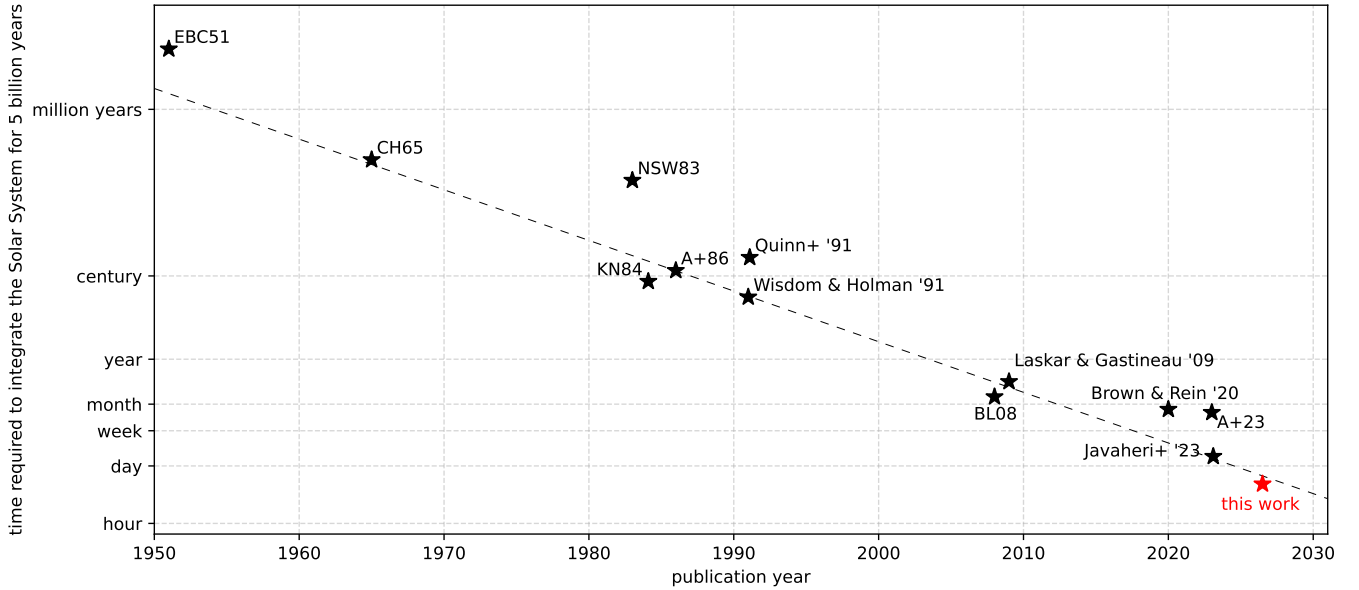


Figure 1. The wall-clock time required for an integration of the Solar System with all eight planets for 5 Gyr as reported by W. J. Eckert et al. (1951); C. J. Cohen & E. C. Hubbard (1965); X. X. Newhall et al. (1983); H. Kinoshita & H. Nakai (1984); J. H. Applegate et al. (1986); T. R. Quinn et al. (1991); J. Wisdom & M. Holman (1991); K. Batygin & G. Laughlin (2008); J. Laskar & M. Gastineau (2009); D. S. Abbot et al. (2023) and P. Javaheri et al. (2023). The bottom right data point shows the result reported in this paper. The four rightmost data points all use a version of either `WHFast` or `WHFast512`. The dashed line shows an exponential function, halving the walltime every 2.5 yrs. The data has been adopted from Sam Hadden, https://shadden.github.io/nbody_history/. A historical N-body scaling figure for large N appears in W. Dehnen & J. I. Read (2011).

age every 2.5 yr. This is consistent with Moore’s law (G. E. Moore 1965), which predicts that the number of transistors on a computer chip doubles approximately every 2 yr (G. E. Moore 1975).

What Fig. 1 hides is that CPU clock rates have barely increased since approximately the year 2000 (A. Danowitz et al. 2012). This is due to physical constraints, such as CPU heat dissipation, and architectural choices that favor parallel processing of large data sets, including multi-core CPUs and GPUs (H. Esmailzadeh et al. 2011). The performance gains shown in Fig. 1 over the last two decades, particularly those reported by P. Javaheri et al. (2023) and the present work, are not due to higher clock rates but rather due to improvements in algorithms and the use of advanced CPU instructions.

AVX512 describes a set of CPU features which extends the standard x86 instruction set by adding new instructions that operate on 512-bit wide registers. Such a register can store up to 8 double precision floating point numbers. One AVX512 instruction can then operate on all 8 floating point numbers in parallel. This makes it ideal for scientific computing where many floating point operations are required. Parallelization with AVX512 has the big advantage that all operations happen on one CPU, thus avoiding the need for communica-

tion between different CPU/GPU cores or cluster nodes. This is particularly important for the kind of astrophysical simulation considered here because one timestep can take as little as 100ns⁶. Some compilers can automatically generate AVX512 instructions, but as we show in this paper, their capabilities are nowhere near what can be achieved by assembling the instructions by hand. The fact that the Solar System has eight planets is a fortunate coincidence as it allows us to fully occupy a register with exactly one floating point number for each of the eight planets. This allows us to speed up simulations of the Solar System particularly well. However, `WHFast512` can be used to integrate other planetary systems as long as the number of planet is less or equal to eight.

Our work builds on that described in P. Javaheri et al. (2023), which uses explicit AVX512 instructions for N-body simulations of planetary systems. This first version of the `WHFast512` integrator was written in C99 using intrinsic functions for the AVX512 instructions. Although this approach is convenient from a programming per-

⁶ This corresponds to a light travel time of only 30m, implying that no computer larger than 30m can possibly be used to speed up the calculation of one timestep. This is a strict causal limit and does not depend on the specific technology used.

spective, it leaves many decisions about register use and the order of operations to the compiler. The present paper describes several significant improvements to the **WHFast512** integrator, which we refer to as **WHFast512** Version 2. Specifically, we reduce the wall-clock time by a factor of 4 while increasing the accuracy by 5 orders of magnitude.

We note that [K. Nitadori et al. \(2006\)](#) and [A. Tanikawa et al. \(2012\)](#) also use SSE and AVX instructions to speed up N-body simulations. Their methods use a Hermite scheme and target star clusters with more particles than the systems considered here, $N \sim 10^3 - 10^5$. Like us, they accelerate their algorithms by writing crucial parts in assembly. Work by [W. Dehnen & D. M. Hernandez \(2017\)](#) also describes some success in using vectorization to speed up large simulations with $N = 1024$.

The plan for this paper is as follows. In Section 2 we describe the algorithmic improvements that we made such as switching to Jacobi coordinates and changes to the Kepler solver and the Stiefel functions. We then focus on the performance optimizations in Section 3 where we discuss our choice to reimplement **WHFast512** in assembly, the advantages of keeping the entire simulation in registers, an efficient matrix multiplication algorithm, how we avoid branching, and when the use of approximate operations is acceptable. Various tests are presented in Section 4 which demonstrate the accuracy and performance of our new integrator. We discuss in Section 5 why we think our integrator is almost optimal, making it very hard to achieve further speed improvements. We conclude in Section 6.

2. ALGORITHMIC IMPROVEMENTS

The main idea of a WH integrator is to split the motion of each planet into two parts: motion around the central object on a Keplerian orbit and perturbations from interactions with the other planets. This perturbative approach reduces the error, relative to a non-perturbative integrator, by a factor comparable to the Jupiter-to-Sun mass ratio, roughly 10^{-3} in the Solar System.

To perform this splitting, a WH integrator must solve two problems in a leap-frog fashion. First, planets are moved along Keplerian orbits in the *Kepler step*. Kepler’s equation has no closed form solution, so this step is performed iteratively. Second, forces from planet-planet interactions are applied in the *interaction step*. This step can be computed explicitly in any inertial frame but can be computationally expensive because it involves $N \cdot (N-1)/2$ force evaluations. For a more comprehensive introduction to the WH integrator see the original

papers by [J. L. Wisdom \(1981\)](#) and [J. Wisdom & M. Holman \(1991\)](#) and specifically for the **WHFast** and previous **WHFast512** implementations of the WH integrator, see [H. Rein & D. Tamayo \(2015\)](#) and [P. Javaheri et al. \(2023\)](#) respectively.

Parallelizing the Kepler and interaction steps poses different challenges and we describe our algorithmic improvements in this section.

2.1. Jacobi Coordinates

The separation into Keplerian motion and a perturbation part is not unique and can be done in different ways, each of which has its own advantages and disadvantages. [H. Rein & D. Tamayo \(2019\)](#) lists the two Hamiltonian splittings most relevant to this paper: DHC and Jacobi coordinates. Other choices exist, including canonical heliocentric coordinates ([J. Laskar & P. Robutel 1995](#)), inertial Cartesian coordinates, barycentric coordinates, and the maps of [D. M. Hernandez & E. Bertschinger \(2015\)](#) as well as [D. M. Hernandez \(2016\)](#). The first version of **WHFast512** used democratic heliocentric coordinates (DHCs) for the Hamiltonian splitting. This was done because it was easier: DHCs do not require coordinate transformations between the interaction and Kepler steps. Furthermore, planets do not need to be ordered. However, for the same timestep, a Hamiltonian splitting based on DHCs is less accurate than one based on Jacobi coordinates ([A. Farrés et al. 2013](#)). In particular, DHCs are not well suited for long-term integrations of planetary systems, as described in detail in [H. Rein et al. \(2026\)](#). This is because the DHC splitting of the Hamiltonian introduces an additional numerical precession that can easily dominate over physical effects. To avoid unphysical results, one needs to reduce the timestep by almost an order of magnitude. However, doing so eliminates the potential speed-up gained by **WHFast512**.

For this reason, we use Jacobi coordinates in the new version of **WHFast512**. The main downside of Jacobi coordinates is that they require coordinate transformations between each Kepler step and interaction step. Specifically, one needs to transform the positions from Jacobi coordinates to heliocentric coordinates before the interaction step and transform the accelerations from heliocentric coordinates to Jacobi coordinates afterward.

One key insight that allows us to implement these coordinate transformations efficiently is that they can be written as linear transformations, specifically 8×8 triangular matrices for an eight-planet system. The matrix coefficients are constants and can be calculated once at the beginning of the simulation because they depend only on the particle masses, not on positions or velocities

(see, e.g., [H. Beust 2003](#)). Our specific implementation of an efficient matrix-vector multiplication is described in Section 3.3.

Hamiltonian splittings with either Jacobi or DHCs admit symplectic correctors, which can reduce errors by orders of magnitude with almost zero overhead ([J. Wisdom et al. 1996](#); [D. M. Hernandez & W. Dehnen 2017](#)). However, with general relativistic corrections enabled (see next section), the [J. Wisdom et al. \(1996\)](#) style correctors in DHCs no longer work. This is another reason for using Jacobi coordinates. We implement symplectic correctors of order 17 using a concatenation of Kepler and interaction steps.

2.2. General Relativity, Close Encounters, and Ejections

The stability of planetary systems, particularly the Solar System, is sensitive to the precise precession frequencies of the planets ([J. Laskar 2008](#); [J. Laskar & M. Gastineau 2009](#); [G. Boué et al. 2012](#); [G. Brown & H. Rein 2023](#)). We therefore implement an option to include a general relativistic correction to Newtonian gravity in the form of an additional $1/r^2$ potential centred on the central object. This first order correction mimics the effects of general relativistic precession ([A. Nobili & I. W. Roxburgh 1986](#)). Because we use Jacobi coordinates, we already calculate the heliocentric distance of each planet in the interaction step. Thus, adding this correction term has minimal cost because no additional square roots or divisions are required. We note that this treatment of GR is only an approximation. It is well suited for cases like the Solar System where accurate secular precession rates are important. However, it introduces small errors in the orbital frequencies and might thus not be sufficient for other cases, in particular in strong gravity regimes.

We also implement optional checks for close encounters and ejections at every timestep. The ejection check compares the heliocentric distance of each planet with a preset value. The simulation stops when the distance exceeds this value. Similarly, the encounter check compares the distance between each pair of planets with a preset value. The simulation stops when the distance falls below this value. Once again, we can reuse distances that are already calculated in the interaction step, so these checks do not add significant overhead.

All three optional features increase the runtime by at most a few percent.

2.3. Kepler Solver

Since no closed form solution to Kepler’s equation exists, this non-linear problem must be solved iteratively.

In the first version of **WHFast512**, an initial guess was refined using a fixed number of iterations. This works for packed systems such as the Solar System because we can assume a priori that the planets occupy only a restricted range of orbital parameters (eccentricity and orbital period), as they would otherwise collide with one another. However, this approach is not always optimal and can fail for some systems, particularly those in which high eccentricities might occur temporarily without leading to an instability. For this reason, we improve the Kepler solver in this version of **WHFast512** to make it more general.

We follow the notation of [S. Mikkola \(1997\)](#) and [H. Rein & D. Tamayo \(2015\)](#) and write Kepler’s equation as

$$r_0 X + \eta_0 G_2(\beta, X) + \zeta_0 G_3(\beta, X) - dt = 0, \quad (1)$$

where G_n are Stiefel functions ([E. L. Stiefel & G. Scheifele 1975](#))

$$G_n(\beta, X) = X^n c_n(\beta X^2). \quad (2)$$

The Stiefel functions in turn depend on Stumpff functions c_n which can be written as a Taylor series (see Eq. 8). The remaining quantities are

$$\beta = \frac{2M}{r_0} - v_0^2, \quad \eta_0 = \mathbf{r}_0 \cdot \mathbf{v}_0, \quad \text{and} \quad \zeta_0 = M - \beta r_0,$$

which depend on the initial positions $\mathbf{r}_0$, velocities $\mathbf{v}_0$, and the standard gravitational parameter⁷, M . The goal of the Kepler solver is to find X given the timestep dt , M , and the positions and velocities at the beginning of the timestep.

Our new algorithm works as follows. We start with a first order guess for $X = dt/r_0$. We then perform two Halley steps to refine X . In the first step, we include only 9 terms in the Taylor expansion of the Stumpff functions. In the second step, we include 11 terms. Now that the estimate is relatively close to the solution, we refine it with a Newton step for which we include 19 terms in the Stumpff functions. At this point we introduce a convergence check, similar to what is present in **WHFast** but is absent in the first version of **WHFast512**. We refine the solution for each planet with Newton iterations until it has converged according to an empirically determined criterion $|\delta X| < \epsilon = 10^{-11}$. We found that if we have not yet converged after four Newton iterations, then we are likely to not converge at all. Thus, if

⁷ We write the standard gravitational parameter as M in this paper because without loss of generality we can set the gravitational constant to one and thus reduce the number of multiplications. See also Section 4.5.

a converged solution is not obtained after the first four Newton iterations, then we switch to a fallback bisection method. Once the solution for X has converged, we calculate the f and g functions:

$$f = M \frac{G_2}{r_0} \quad \dot{f} = \frac{M G_1}{r_0 r} \quad (3)$$

$$g = dt - M G_3 \quad \dot{g} = \frac{M G_2}{r}, \quad (4)$$

where r is the distance at the end of the timestep, $r = r_0 + \eta_0 G_1 + \zeta_0 G_2$. Note that our definitions of f , $\dot{f}$, and $\dot{g}$ (but not g) differ from the definitions in [S. Mikkola \(1997\)](#). Using our definitions, the positions and velocities at the end of the timestep can be written as

$$\mathbf{r} = (1 - f)\mathbf{r}_0 + g\mathbf{v}_0, \quad \mathbf{v} = -\dot{f}\mathbf{r}_0 + (1 - \dot{g})\mathbf{v}_0. \quad (5)$$

Our definitions of f , g , $\dot{f}$, and $\dot{g}$ allow us to use more efficient and accurate fused multiply-add instructions in this final step and write it as

$$\mathbf{r} = \text{fma}(g, \mathbf{v}_0, \text{fnma}(f, \mathbf{r}_0, \mathbf{r}_0)) \quad (6)$$

$$\mathbf{v} = \text{fnma}(\dot{f}, \mathbf{r}_0, \text{fnma}(\dot{g}, \mathbf{v}_0, \mathbf{v}_0)) \quad (7)$$

where $\text{fma}(a, b, c) = ab + c$ and $\text{fnma}(a, b, c) = -ab + c$.

With AVX512, we can perform the Kepler step in parallel for eight planets. However, different planets might require a different number of iterations to converge, with typically the innermost or most eccentric planet requiring the most. To ensure that each planet's solution remains independent of all other planets in the simulation, only the lanes for planets that have not yet converged are updated at each iteration and planets that have already converged are no longer modified. This is important because an excessive number of iterations on an already converged solution can actually worsen the accuracy due to accumulating roundoff errors. We achieve this using masked instructions that write only to the lanes corresponding to planets that have not yet converged. Masked instructions themselves have no overhead in terms of latency and throughput, but we require some CPU cycles to generate and update the masks at each iteration.

The specific sequence of Halley and Newton steps and the number of terms used in the Taylor series expansion of the Stumpff functions were identified through a brute-force comparison of all possible combinations, subject to the requirement of convergence to machine precision. We then chose the most efficient sequence for what we consider a typical integration (see Appendix B for more details). For different applications, for example in cases with highly eccentric orbits, a different sequence might be slightly faster.

To summarize our implementation of the Kepler solver, we list the pseudocode in Algorithm 1. The Kepler solver is evaluated for all 8 planets in parallel using AVX512 instructions. Note that for readability, we do not show the masked instructions or the mask generation.

Input: dt , positions, velocities
Output: positions, velocities
 $r, \beta, \eta, \zeta \leftarrow$ from positions and velocities
/* First order guess ***/**
 $X \leftarrow dt/r$
/* First two refinements ***/**
 $G_{0,1,2,3} \leftarrow \text{Stiefel}(X, \beta, \text{nTerms} = 9)$
 $X \leftarrow \text{Halley}(X, dt, G_{0,1,2,3}, \beta, \eta)$
 $G_{0,1,2,3} \leftarrow \text{Stiefel}(X, \beta, \text{nTerms} = 11)$
 $X \leftarrow \text{Halley}(X, dt, G_{0,1,2,3}, \beta, \eta)$
/* Newton iteration loop ***/**
 $X' \leftarrow X$
for $i \leftarrow 0$ **to** 4 **do**
 $G_{1,2,3} \leftarrow \text{Stiefel}(X, \beta, \text{nTerms} = 19)$
 $X \leftarrow \text{Newton}(X, dt, G_{1,2,3}, \beta, \eta)$
 if $|X - X'| < \epsilon$ **then**
 goto converged
 $X' \leftarrow X$
/* Fallback bisection method ***/**
Period $\leftarrow$ from positions and velocities
 $X_{\min} \leftarrow 2\pi \cdot \text{floor}(dt/\text{Period})/\sqrt{\beta}$
 $X_{\max} \leftarrow X_{\min} + 2\pi/\sqrt{\beta}$
 $X \leftarrow (X_{\min} + X_{\max})/2$
for $i \leftarrow 0$ **to** 51 **do**
 $G_{1,2,3} \leftarrow \text{Stiefel}(X, \beta, \text{nTerms} = 19)$
 if $r \cdot X + \eta \cdot G_2 + \zeta \cdot G_3 < dt$ **then**
 $X_{\min} \leftarrow X$
 else
 $X_{\max} \leftarrow X$
 $X \leftarrow (X_{\min} + X_{\max})/2$
/* Final update ***/**
converged:
 $G_{1,2,3} \leftarrow \text{StiefelCompensated}(X, \beta, \text{nTerms} = 19)$
 $f, g, \dot{f}, \dot{g} \leftarrow$ using $G_{1,2,3}, \eta, \zeta, dt$
new positions $\leftarrow$ using f, g , positions, velocities
velocities $\leftarrow$ using $\dot{f}, \dot{g}$, positions, velocities
positions $\leftarrow$ new positions

Algorithm 1. Pseudocode of the new **WHFast512** Kepler solver. $G_{0,1,2,3}$ are the Stiefel functions defined in [S. Mikkola \(1997\)](#) and implemented using Algorithm 2. The quantities X , β , η , and ζ are defined in [H. Rein & D. Tamayo \(2015\)](#).

2.4. Roundoff in the Stiefel functions

We need to evaluate Stiefel functions G_n for the Kepler solver described above. The Stiefel functions in turn depend on the Stumpff functions c_n . We calculate the

Stumpff functions from a truncated Taylor series:

$$c_n(z) \equiv \sum_{j=0}^{n_{\text{terms}}} \frac{(-z)^j}{(n+2j)!}. \quad (8)$$

In floating point arithmetic, this evaluation carries a small but systematic bias. The bias has two different origins. First, the inverse factorial coefficients of the series are pre-calculated and rounded to the nearest representable number. Second, the summation of the series involves rounding at every term. Both effects are tiny, of the order of 10^{-20} per timestep, but they can accumulate over time because the rounding errors in the coefficients are constant. This causes the energy error in long integrations to grow linearly in time.

This bias is present in the first version of **WHFast512**, although it is hardly visible in a typical simulation because the overall scheme error is significantly larger than in our new version (see Section 2.1). Here, we aim to implement a less biased scheme in which the energy error grows only with the square root of the number of steps, consistent with Brouwer’s law (D. Brouwer 1937).

We reduce the bias with the following two changes. First, we precalculate the rounding errors of the inverse factorial coefficients of the series in extended precision. Second, we evaluate the series with a compensated summation that includes both the original inverse factorial coefficients and rounding error corrections. The compensated summation retains the low order bits that would otherwise be lost. Neither change is sufficient on its own because the coefficient corrections are too small to be represented in double precision and are simply discarded unless the compensated summation also carries the extra bits. We test the effectiveness of both changes together as well as individually in Appendix C. Only the last few terms of the series require this correction because, for a typical timestep, the correction is small and the higher order terms are negligible. Furthermore, only the final evaluation of c_n that we ultimately use for the f and g functions requires this correction because this evaluation advances the particles’ positions and velocities. Intermediate evaluations used inside the iterations of the Kepler solver do not significantly affect the final state. The correction therefore adds only a handful of operations to the series evaluation per timestep, but it restores the expected random walk behaviour of the energy error over multi-Gyr integrations, as we show in Section 4.3.

We list the pseudocode for our Stiefel function evaluation in Algorithm 2. The Stiefel functions are evaluated for all 8 planets in parallel using AVX512 instructions.

Input: $X, \beta, n_{\text{Terms}}$
Output: $G_{0,1,2,3}$
 $z \leftarrow \beta \cdot X^2$
 /* Stumpff functions by Horner’s method */
 $c_2 \leftarrow 1/(n_{\text{Terms}} - 1)!$
 $c_3 \leftarrow 1/n_{\text{Terms}}!$
 /* Plain Horner for the higher order terms */
for $k \leftarrow n_{\text{Terms}} - 2, n_{\text{Terms}} - 4, \dots, 7$ **do**
 $c_3 \leftarrow 1/k! - z \cdot c_3$
 $c_2 \leftarrow 1/(k-1)! - z \cdot c_2$
 /* Compensated Horner for the last two terms */
 $d_2 \leftarrow 0$
 $d_3 \leftarrow 0$
for $k \leftarrow 5, 3$ **do**
 $p \leftarrow z \cdot c_3$
 $p_e \leftarrow \text{fma}(z, c_3, -p)$
 $c_3 \leftarrow 1/k! - p$
 $s_e \leftarrow (1/k! - c_3) - p$
 $d_3 \leftarrow (s_e - \delta_k - p_e) - z \cdot d_3$
 $p \leftarrow z \cdot c_2$
 $p_e \leftarrow \text{fma}(z, c_2, -p)$
 $c_2 \leftarrow 1/(k-1)! - p$
 $s_e \leftarrow (1/(k-1)! - c_2) - p$
 $d_2 \leftarrow (s_e - \delta_{k-1} - p_e) - z \cdot d_2$
 $c_2 \leftarrow c_2 + d_2$
 $c_3 \leftarrow c_3 + d_3$
 /* Stiefel functions from Stumpff functions */
 $G_2 \leftarrow X^2 \cdot c_2$
 $G_3 \leftarrow X^3 \cdot c_3$
 $G_1 \leftarrow X - \beta \cdot G_3$
 $G_0 \leftarrow 1 - \beta \cdot G_2$

Algorithm 2. Pseudocode of our Stiefel function evaluation. The Stumpff functions c_2 and c_3 are evaluated by Horner’s method from their truncated Taylor series, and the Stiefel functions follow from the recurrence relations (S. Mikkola 1997). For the last two terms that appear in Horner’s method, compensated summation is used to accumulate the rounding of every operation in a low order word d , which is added back at the end. Here, $\text{fma}(a, b, c) = ab + c$ is evaluated with a single rounding, and δ_k is the precomputed rounding error of the coefficient $1/k!$. This compensation is applied only in the final evaluation that produces the f and g functions; intermediate evaluations use plain Horner for all terms.

3. OPTIMIZATIONS

In Section 2 we described algorithmic improvements to **WHFast512**. In this section we focus on the implementation details that make this algorithm run fast.

3.1. Assembly

The first version of **WHFast512** was written in C99 with special intrinsics that map directly to AVX512 instructions. This approach makes it easy to include AVX512 in existing code. However, one disadvantage

is that it does not allow fine-grained control over various aspects of the code because intrinsics leave those details to the compiler. For example, the compiler allocates registers and decides which variables to put on the stack.

For these reasons, the main parts of the new version of **WHFast512** are now written in about 800 lines of x86 assembly. We make extensive use of macros, which allow us to reuse much of the code for different configurations. Specifically, macros generate 24 slightly different functions for simulations with different number of planets, with and without general relativistic corrections, with and without encounter checks, as well as with and without ejection checks (see Section 2.2). Similarly, macros create six different functions for symplectic correctors. This approach allows us to provide highly optimized functions for a variety of use cases while avoiding branching because we can select the most appropriate kernel once at the beginning of the integration.

3.2. Keeping the Simulation in Registers

AVX512 supports 32 CPU registers, each 512 bits wide. Using only seven registers, we can keep the entire simulation state, specifically the masses, positions, and velocities of all particles, in registers at all times. The remaining registers are sufficient for performing all computations and storing intermediate values and constants such as the timestep and numerical factors.

We still need to use memory to store some constants, such as the 128 matrix coefficients (see below) as these alone would take up half the available registers. However, by keeping the simulation state in registers, we never have to write to memory during an integration except when an output is required. Thus, the cache is not polluted and can work very efficiently. In fact, we have zero cache misses⁸. As a result, **WHFast512**'s performance is not limited by the speed of memory access in contrast to many other HPC applications.

3.3. Matrix Multiplications

As discussed in Section 2.1, we need to perform several matrix multiplications to transform from Jacobi coordinates to heliocentric coordinates and back. In the case of eight planets, we need to multiply an 8×8 matrix by an 8-element vector containing x , y , or z coordinates. Thus, in total, we need to perform six matrix-vector multiplications per timestep. It is therefore crucial to optimize this calculation as much as possible. We achieve very good performance by combining several approaches.

First, the matrix elements are constants stored in memory. Because we never write to memory (see above), they remain in the CPU's L1 cache, and load operations are therefore very efficient.

Second, we perform the multiplications for three coordinates at the same time. This way, we need to load each matrix element only once and can reuse it for all three coordinates.

Third, we use two accumulators per coordinate. The accumulators are only combined at the end of the multiplication. This allows the CPU to schedule instructions in parallel and avoids data dependencies in the pipeline.

Fourth, we use fused multiply-add instructions extensively. These instructions improve both throughput and accuracy by avoiding an intermediate rounding step.

3.4. Avoiding Branches

Branches such as those originating from conditional statements or loops can significantly slow down a simulation. This is because modern CPUs often execute instructions out-of-order, but branches with multiple possible outcomes make such execution more difficult. We therefore carefully remove almost all branches in **WHFast512**'s kernel and are left with only two.

The first branch controls the overall loop over individual timesteps. This branch allows the loop to terminate after the specified number of timesteps. The other remaining branch is in the Kepler step, where it checks for convergence (see Section 2.3).

3.5. Square Roots

The interaction step is dominated by the pairwise force evaluations between planets, each of which is proportional to $1/r^3$, where r is the distance between the two particles. No single instruction computes $1/r^3$, and on current CPUs and both the square root and the division instruction have a low throughput and high latency. We therefore use the approximate reciprocal square root instruction **rsqrt14**, which returns an estimate of $1/\sqrt{x}$ accurate to about 14 bits. The **rsqrt14** instruction has a latency of about 8 clock cycles compared to over 50 for the normal square root plus division instructions. We already have r^2 from the distance calculation, so applying **rsqrt14** to r^2 returns an estimate of $1/\sqrt{r^2} = 1/r$. Similar to A. Tanikawa et al. (2012), we set $a = r^2$ and use Newton-Raphson iterations to solve $f(y) = y^{-2} - a = 0$, whose positive root is $y = 1/\sqrt{a} = 1/r$. Starting from the **rsqrt14** estimate y_0 , each iteration is $y_{n+1} = y_n(3/2 - ay_n^2/2)$. We then obtain $1/r^3 = y_2^3$ with two additional multiplications. Doing so makes the reciprocal square root accurate to machine precision. This procedure replaces

⁸ Cache misses might still occur if the operating system performs some other operation on the CPU during an integration.

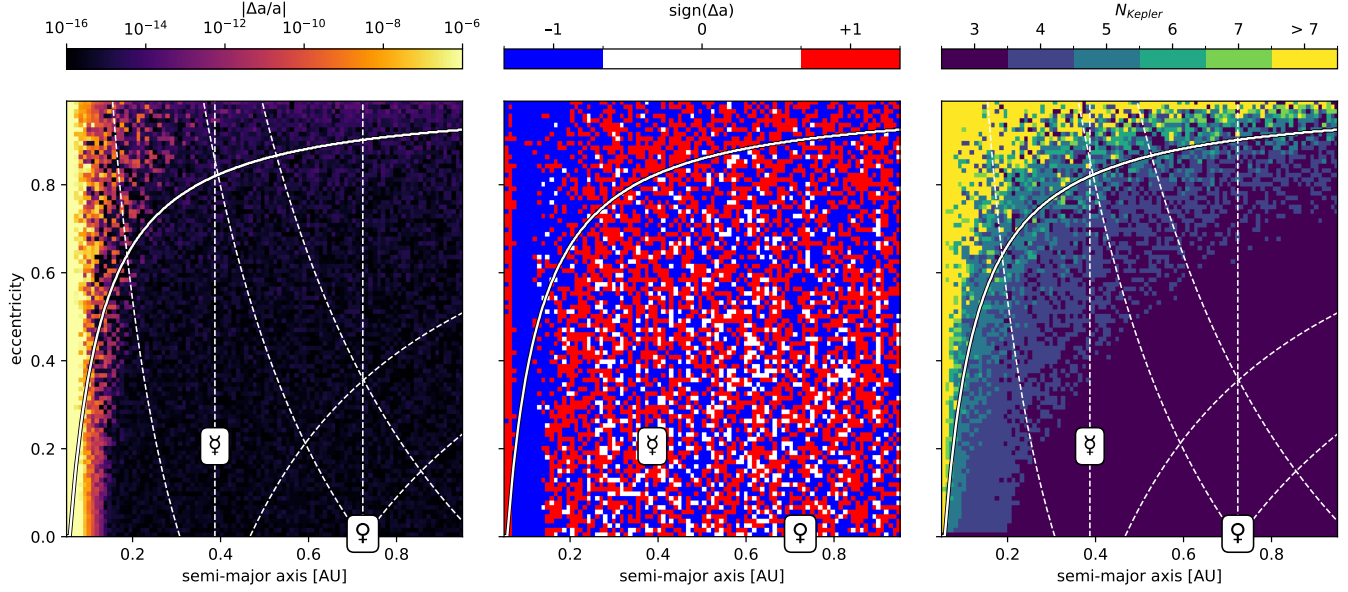


Figure 2. The absolute relative semi-major axis error, the sign of the semi-major axis error, and the number of iterations in the Kepler solver. Here, a test particle is integrated for one timestep of 5 days. The current locations of the Solar System planets are shown in each plot. See the text for a description of the lines.

the expensive square root and the division instructions with a short sequence of multiply and fused-multiply-add instructions, all of which are high throughput and low latency.

In comparison to the actual IEEE 754 compliant square root and division instructions, the result of our procedure is not guaranteed to be correctly rounded. However, in the case of the interaction step, the rounding is not crucial as a) the accelerations we calculate are small to begin with as they are perturbations to the Keplerian motion, b) because they get multiplied by a small number, the timestep, and c) because we encounter the same rounding error for both particles and thus, thanks to Newton’s third law, energy is conserved regardless.

To illustrate the last point, let us consider the interaction between two planets at a distance r from each other. Suppose we calculate everything exactly but incorrectly round the inverse square root to $[1/\sqrt{r}]_{\text{fp}} = 1/\sqrt{r} + \delta$ where $[\cdot]_{\text{fp}}$ represents our approximation and δ is the error. Then, with the exception of the sign, the error in the force felt by both planets is exactly the same as long as we use the same approximation $[1/\sqrt{r}]_{\text{fp}}$ for both planets. A different way to interpret this is that we actually calculate forces without *any* error (and most importantly ensuring $F_{ij} = F_{ji}$), just for a slightly wrong force law.

In the Kepler step we need both the distance r as well as its reciprocal $1/r$: r is used to compute ζ , and $1/r$ to compute β and the first order guess for X . Here

the square root is evaluated only once per planet and timestep, and not once for each of the $N \cdot (N-1)/2$ pairs as in the interaction step. It therefore does not lie on the throughput-critical path, and the Kepler step remains dominated by the later iterative solver. Furthermore, because the result of the square root operation is used at the end of the Kepler step to advance the positions and velocities, any bias would propagate to those quantities. We thus avoid the `rsqrt14` instruction used for the pairwise forces and use the IEEE 754 compliant square root and division instructions in the Kepler step instead.

4. TESTS

4.1. Kepler solver

The Kepler solver of `WHFast512` is crucial for both the speed and accuracy of the entire integrator. We therefore test the Kepler solver extensively with one particle at a time. We use a fixed timestep of 5 days and plot the results over a grid of semi-major axis and eccentricity. The phase of the orbit is chosen randomly. Because gravity is scale invariant, our choice of a timestep of 5 days is somewhat arbitrary and we chose it simply because similar values have been used in past simulations of the Solar System. What actually matters is the ratio of the timestep over the orbital period. Because of that the results of this test can be applied to any other planetary system.

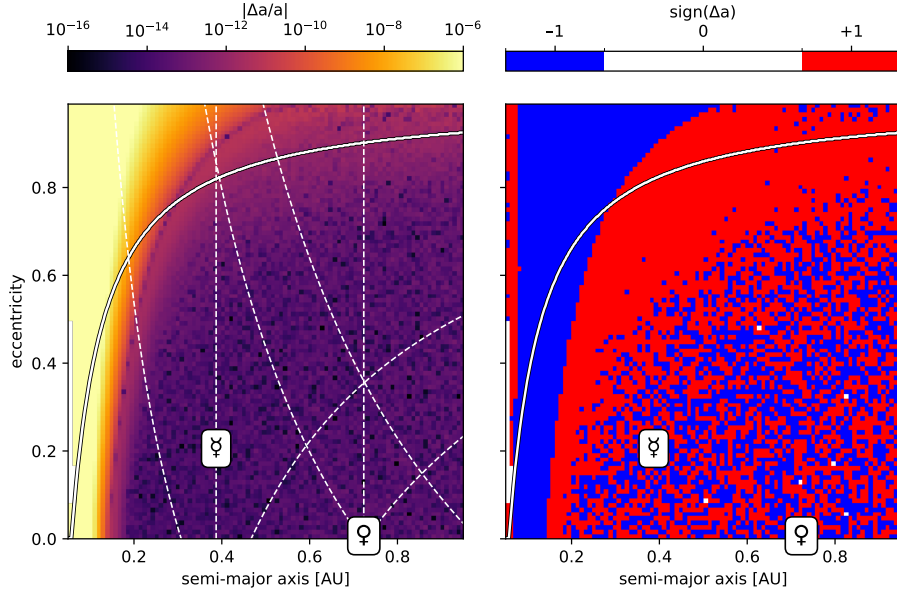


Figure 3. Same as the first two panels of Fig. 2, but each particle is integrated for 1000 orbital periods. The top and left of the plots show a biased error growth. The bottom right of the plots show an unbiased error growth in the most important area of parameter space.

Figure 2 shows the relative semi-major axis error, the sign of the semi-major axis error, as well as the number of iterations in the Kepler solver. We also overplot the current orbital parameters of Mercury and Venus. The vertical dashed lines correspond to the planets’ semi-major axes. The curved dashed lines can be used to find regions in parameter space in which a particle crosses the orbit of a Solar System planet on its present-day orbit. For example, if the eccentricity of Mercury rises to ~ 0.85 , then its orbit will cross the present-day orbit of Venus. Similarly, if the eccentricity of Venus rises to 0.4, then it will cross orbits with both Mercury and Earth. The solid white line shows where the pericenter timescale is equal to the timestep (see J. Wisdom 2015 for a definition of the pericenter timescale). These lines help to understand whether the accuracy in a region of parameter space matters or not. For example, if the timestep is larger than the pericenter timescale (top left above the solid white line), then we most definitely fail to resolve physically relevant timescales⁹. Similarly, if a planet becomes very eccentric, then the planet is likely to have a close encounter with a neighbouring planet shortly afterwards. In both cases, the Kepler solver is no longer the limiting factor for the accuracy of a simulation.

⁹ J. Wisdom (2015) finds that at least 17 timesteps per pericenter timescale are required for some systems.

The left panel of Fig. 2 shows that, in the most relevant region of parameter space, the Kepler solver is accurate to machine precision, $\sim 10^{-16}$. The middle panel shows that the sign of the error is random in the same region. This is important for having an unbiased integrator (see Section 2.3 and Appendix C). For example, imagine we have an integrator accurate to 10^{-16} after one timestep but with a bias of the same magnitude (i.e., the sign of the error is the same every time). After, say, 10^{11} steps corresponding to a few Gyr in a Solar System integration, the error in the semi-major axis would have increased to 10^{-5} from this error source alone. If, on the other hand, the integrator is unbiased, then the error would only be $10^{-16} \cdot 10^{11/2} \approx 3 \cdot 10^{-11}$.

The right panel shows the number of iterations required for the Kepler solver to converge. Since we always have two Halley steps followed by at least one Newton step, the minimum number of iterations is 3. For a large region of parameter space, we indeed converge in only 3 iterations. More iterations are required for higher eccentricities and shorter orbital periods (we show additional tests for bound orbits with extremely high eccentricities in Appendix A). The yellow colour indicates where we fail to converge in 7 iterations and thus have to fall back to the bisection method. This only occurs for very eccentric orbits. Note that even in the case where the bisection method was used, the error remains close to machine precision and the sign of the error remains random.

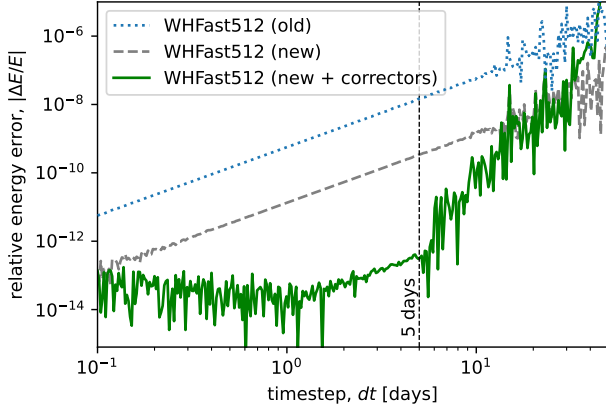


Figure 4. Relative energy error as a function of the timestep for different integrators in a 100 year simulation of the Solar System with all eight planets.

The Kepler solver fails for very short orbital periods. This is expected because our algorithm does not support timesteps that are larger than the orbital period. Fixing this is easy enough to do (it is done in **WHFast**), but would introduce an additional branch in the code which would affect the speed of the algorithm. Since simulations in which the timesteps are large compared to the innermost planet’s period lead to unphysical results anyway, we decided to not implement a Kepler solver which unconditionally converges in all cases.

We next repeat the tests from Fig. 2 with the same 5 day timestep but now integrate each particle for 1000 orbital periods instead of only one timestep. The results are shown in Fig. 3. As expected, the relative semi-major axis error is now larger as errors accumulate. The sign of the semi-major axis error is still random in the bottom right region of parameter space, indicating that the Kepler solver remains unbiased here. However, the area in which the sign is consistently positive or negative has increased. This result reveals a small but biased contribution to the error. As this term grows $\propto t$, it will dominate over the unbiased $\propto \sqrt{t}$ term over sufficiently long timescales. We show in Section 4.3 that the timescale over which this bias becomes apparent in a simulation of the Solar System is longer than the age of the Solar System.

4.2. Convergence

We test the convergence of **WHFast512** by integrating the Solar System with all eight planets for 100 years. The relative energy error at the end of the integration is plotted as a function of the timestep in Fig. 4 for different integrators.

The new **WHFast512** integrator converges as the timestep gets smaller, as expected for a second order

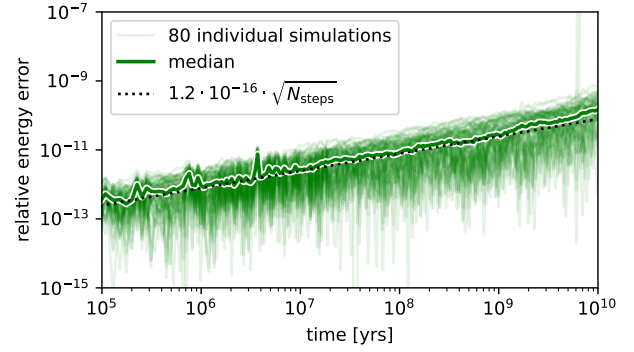


Figure 5. The relative energy error as a function of time in 80 simulations of the Solar System. The bold green line shows the median. The dotted line shows the expected behaviour for an unbiased integrator that is accurate to machine precision after one timestep. Two simulations go unstable at late times, leading to a large energy error.

scheme. When symplectic correctors are turned on, the errors are reduced by a further factor of $\sim 10^3$ for timesteps ~ 5 days. For timesteps smaller than ~ 2 days, the error starts to increase again as the timestep is reduced further because we reach the floor imposed by double precision floating point arithmetic.

Note that the old version of **WHFast512** performs significantly worse because it uses democratic heliocentric coordinates rather than Jacobi coordinates and does not support symplectic correctors. For the same timestep, the new version of **WHFast512** is 5 orders of magnitude more accurate.

4.3. Long-term Integrations

We test the long-term conservation properties of **WHFast512** by simulating the Solar System with all eight planets and general relativistic corrections for 10 Gyr. We use a timestep of 6 days and turn on symplectic correctors. The initial conditions of each simulation are perturbed by a few meters in a random direction. The relative energy errors of the 80 simulations and their median are shown as a function of time in Fig. 5.

The median relative energy error remains below 10^{-10} over 10 Gyr. The errors grow as $\Delta E/E \sim 1.2 \cdot 10^{-16} \cdot \sqrt{N_{\text{steps}}}$. This shows that the integrator is unbiased and follows Brouwer’s law (D. Brouwer 1937) over physically relevant timescales. We expect that over much longer timescales, we will eventually see linear error growth (see the discussion in Section 4.1).

Note that two simulations go unstable at late times which leads to a spike in their energy errors. Instabilities in the Solar System are expected and the rate of instability is consistent with previous results (J. Laskar & M. Gastineau 2009; H. Rein et al. 2026).

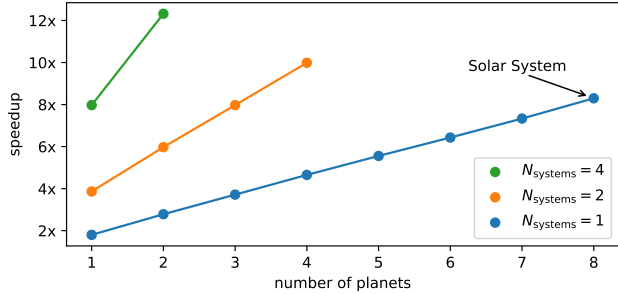


Figure 6. The speedup of WHFast512 compared to WHFast in simulations with different number of planets. For WHFast512 simulations that use $N_{\text{systems}} > 1$, the corresponding WHFast simulation are run sequentially.

4.4. Runtime

To measure the runtime, we compare WHFast512 to WHFast, one of the fastest publicly available N-body integrators for planetary systems (P. Javaheri et al. 2023). Note that WHFast has not been manually optimized for AVX512 but a compiler could in principle try to optimize it with AVX512 instructions automatically. We once again use the Solar System with all 8 planets, a 6 day timestep, and general relativistic corrections as the canonical test case. All results presented here are from simulations performed on an AMD EPYC 9655 2.6GHz processor. Similar speedups were measured on an Intel Xeon Gold 6148 2.4GHz CPU.

We were able to finish a 5 Gyr integration in only 8.7 hours. This corresponds to a 60% speedup when compared to a simulation with the earlier version of WHFast512 using the same timestep and CPU. When compared to the non-SIMD version of WHFast, this corresponds to a more than 8 $\times$ speedup. For WHFast, aggressive compiler optimizations were used (`-O3`, `-march-native`). For WHFast512 the specific compiler or compiler optimizations do not matter as it is written in assembly.

If the number of planets N is ≤ 4 WHFast512 can integrate multiple systems at the same time as we can pack one AVX512 register with data from up to 8 planets and they don’t necessarily need to be in the same system. The speedup for different numbers of planets and different numbers of planetary systems integrated at the same time are shown in Fig. 6. As one can see, the speedup is linear in the number of planets if the number of systems is kept fixed. WHFast512 performs best when 8 planets are integrated at the same time as this fills up one AVX512 register. For the case of 2 planets in 4 different planetary systems, WHFast512 is more than 12 times faster than WHFast.

4.5. A Note on Units

In the tests in this section, we use physical units to describe the setup and results. For example, we use a timestep of 5 days. Since gravity is scale invariant, this does not lead to a loss of generality. What matters physically is the timestep relative to the orbital period.

However, we introduce a scale into the problem by setting the Kepler solver’s convergence criterion to a fixed value ϵ , which has units of time divided by length. In practice, this means that the units should be chosen such that positions and velocities are not excessively large or small. For example, having the innermost planet at 0.01 AU or 100 AU is perfectly fine, but having a planet at 10^{11}m or 10^{-12}Mpc is not. If necessary, ϵ can be adjusted accordingly.

Furthermore, WHFast512 is hardcoded to use units in which the gravitational constant is equal to one. This allows us to avoid multiplications with the gravitational constant entirely. We also hardcode the speed of light when general relativistic corrections are used. Thus, in practice and without loss of generality, the code units used by WHFast512 measure length in astronomical units and time in $\text{yr}/2\pi$.

5. DISCUSSION

We argue that, with the improvements described in Sections 2 and 3, we approach the optimal performance attainable by a Wisdom-Holman type integrator on a present-day high-end CPU. There are several reasons why further significant improvements (speedups of $\gtrsim 2\times$) will be hard to achieve.

First, the code is entirely compute bound. Thus, the latency and throughput of instructions, particularly floating point operations, are the limiting factors for WHFast512. Since we do not use memory to store the simulation state and read only a small amount of memory for constants and coordinate transformations, the code is never memory bound. This is in contrast to many other scientific HPC applications where memory access is the main bottleneck. As a result, higher memory bandwidth, unified memory, in-memory processing, and other memory related technologies are irrelevant to our application.

Second, if we optimize one part of the algorithm, we are limited by the fraction of the total runtime contributed by that part (Amdahl’s argument, G. M. Amdahl 1967). In our case, the main parts of the algorithm are the interaction step, the Kepler step, and the matrix-vector multiplications used for coordinate transformations. These take up approximately 25%, 63%, and 11% of the total runtime, respectively.

Thus, the Kepler step is the most promising target for further optimization. Within the Kepler step, about 50% of the time is spent in the iterations solving Kepler’s equation. As we have shown in Section 4.1, we require only three iterations to converge to a solution for most orbits. We must perform at least two iterations to check for convergence. Thus, we can remove at most one of the three iterations, for example by finding a better initial guess, using data from previous timesteps, or using a higher order solver. Even under the optimistic assumption that such a change would not involve any additional computational work, this would give us a speedup of at most $2/3 \cdot 50\% \cdot 63\% \approx 20\%$. Some improvements might be possible with a different formulation of the Kepler problem itself, for example one that does not rely on the Stiefel functions or the f and g functions.

Next, we consider the interaction step. Here, the main bottlenecks are the throughput and latency of the square root and division instructions needed for the force calculations. We already use the faster 14 bit accurate `rsqrt14` instruction with only two refinement steps. Thus, without hardware changes, such as providing more floating point units for each CPU core, we do not expect a significant speedup of the interaction step.

Finally, even a substantial improvement to the matrix-vector multiplication would yield an overall speedup of at most 11%.

For all these reasons, we believe that our implementation of **WHFast512** is almost optimal for simulations of planetary systems with $N \leq 8$ planets on the current generation of high-end CPUs.

6. CONCLUSIONS

In this paper, we have presented a new version of the **WHFast512** N -body integrator. This Wisdom-Holman style algorithm has been highly optimized for planetary systems with $N \leq 8$ planets. It is by far the fastest N -body integrator currently available for this kind of simulation. Using a timestep of $dt = 6$ days, **WHFast512** can integrate the eight planets of the Solar System for 5 Gyr in under 8.7 hours on an AMD EPYC 9655 processor. On the same hardware, this is 60% faster than the previous version of **WHFast512** and more than $8\times$ faster than the non-SIMD version of **WHFast**, which is itself already significantly faster than most N -body codes (P. Javaheri et al. 2023).

With an almost order of magnitude speed-up, one can explore planetary systems over much longer timescales than what was possible before. For example, we are now in a position where it is feasible to repeat the secular integrations of N. H. Hoang et al. (2022) which span

100 Gyr to investigate the dynamical half-life of Mercury with a direct N -body integration that takes just one week.

In an alternative scenario, we can take advantage of the speed-up by running an order of magnitude more simulations than before for the same computational cost. This allows for much finer parameter space surveys and could for example significantly improve training datasets for machine learning models (D. Tamayo et al. 2020).

We achieved the reported speedup by switching from C to assembly, giving us much finer control over individual registers and CPU instructions. Specifically, this rewrite allowed us to keep the entire simulation state in CPU registers, reduce the number of branches in the code to just two per timestep, and implement a highly optimized matrix-vector multiplication for the coordinate transformations.

In addition to a significant speedup, we also switched **WHFast512** to Jacobi coordinates, allowing us to use symplectic correctors and GR corrections together and thus suppress errors by up to 5 orders of magnitude. We took great care to make our implementation of the Kepler solver as bias-free as possible, thereby enabling reliable long-term integrations.

Although our discussion has focused on the Solar System as the canonical test case, **WHFast512** is a reliable and fast N -body integrator for many small- N simulations. It is particularly well suited for simulations that determine the stability of planetary systems because these simulations are typically stopped as soon as orbits cross or close encounters occur.

We argue that **WHFast512** is almost optimal in terms of performance and accuracy for these kind of simulations on present-day CPUs. Small improvements might still be possible, but the two main bottlenecks are now the square root calculations in the interaction step and the finite number of iterations (~ 3) in the Kepler solver. On the hardware side, the only changes that would improve the runtime are an increase in clock speed or an increase in floating point execution units. Neither of those changes seem likely given the current trend towards high parallelism and low precision aimed at machine learning applications.

Future work might focus on applying similar techniques to those presented in this paper to larger, arbitrary- N simulations and to non-symplectic or hybrid-symplectic integrators that are capable of accurately resolving close encounters as well as hyperbolic orbits. Some applications could also benefit from implementations of additional physics models such as tidal forces and moons, or from better models for general relativistic corrections.

WHFast512 is freely available in the REBOUND integrator package at <https://rebound.hanno-rein.de>. REBOUND is compiled with WHFast512 by default on 64-bit x86 systems. At runtime, REBOUND checks for the relevant CPU flags and reports an error if the CPU does not support AVX512 instructions. WHFast512 is also available in precompiled python wheels for Linux and Windows.

Daniel Tamayo and Dang Pham for helpful discussions and Dang Pham for help with testing WHFast512 on Windows. This research has been supported by the Natural Sciences and Engineering Research Council (NSERC) Discovery Grant RGPIN-2020-04513. The authors did not use generative AI for code generation, or writing and editing this paper.

ACKNOWLEDGMENTS

We thank an anonymous referee for helpful comments which significantly improved the manuscript. We thank

APPENDIX

A. KEPLER SOLVER FOR VERY HIGH ECCENTRICITIES

We here test the Kepler solver for very high eccentricities up to $e = 1 - 10^{-6}$. The results are plotted in Fig 7 and show that the Kepler solver is able to accurately solve the problem even for these extreme eccentricities as long as the timestep is shorter than about a tenth of the orbital period. For the majority of the parameter space shown in Fig. 7 the Kepler solver falls back to the bisection method as can be seen in the right panel. The loss of precision at very high eccentricities is expected and not a shortcoming of the Kepler solver. It is due to finite floating point precision limits: A planet's position and velocity now vary by many orders of magnitude during one orbit and finite floating point precision thus leads to a large error in the *absolute* cartesian position and velocity.

We point out that the parameter space tested here is in general not relevant for simulations of planetary systems if the goal is to determine a stable system's evolution or whether a system is unstable or not. Further note that a standard Wisdom-Holman integrator like WHFast512 might not be the right choice for systems with such high eccentricities because to resolve the shortest dynamical timescale in the problem (the pericenter timescale) one would need a prohibitively small timestep.

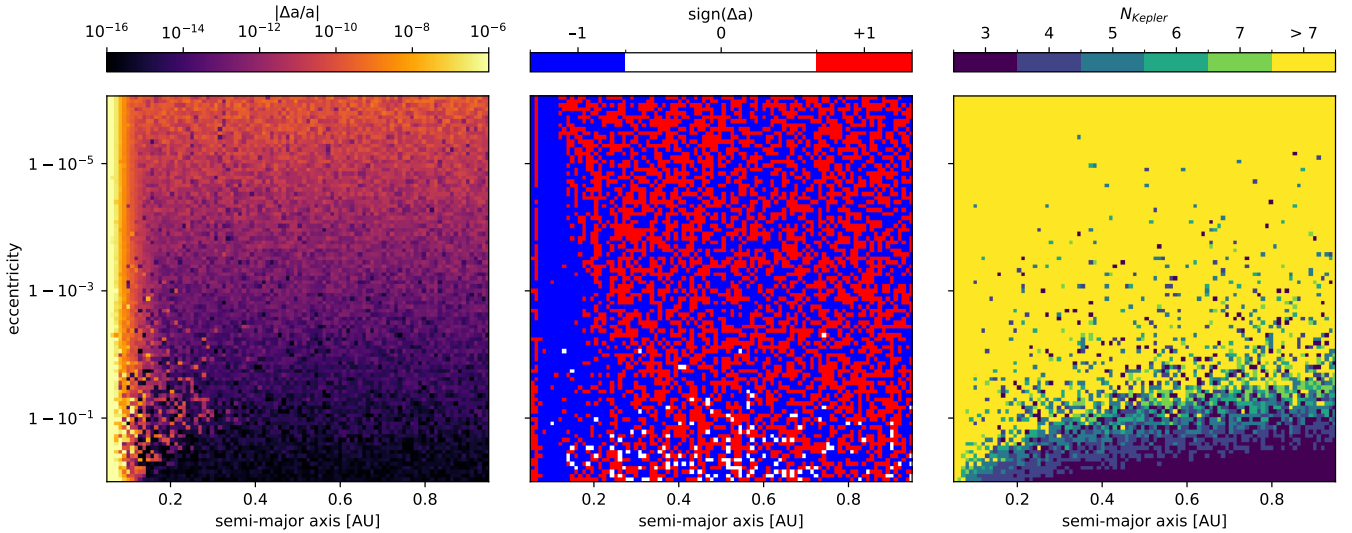


Figure 7. Same as Fig. 2, but for very high eccentricities (note the logarithmic scale).

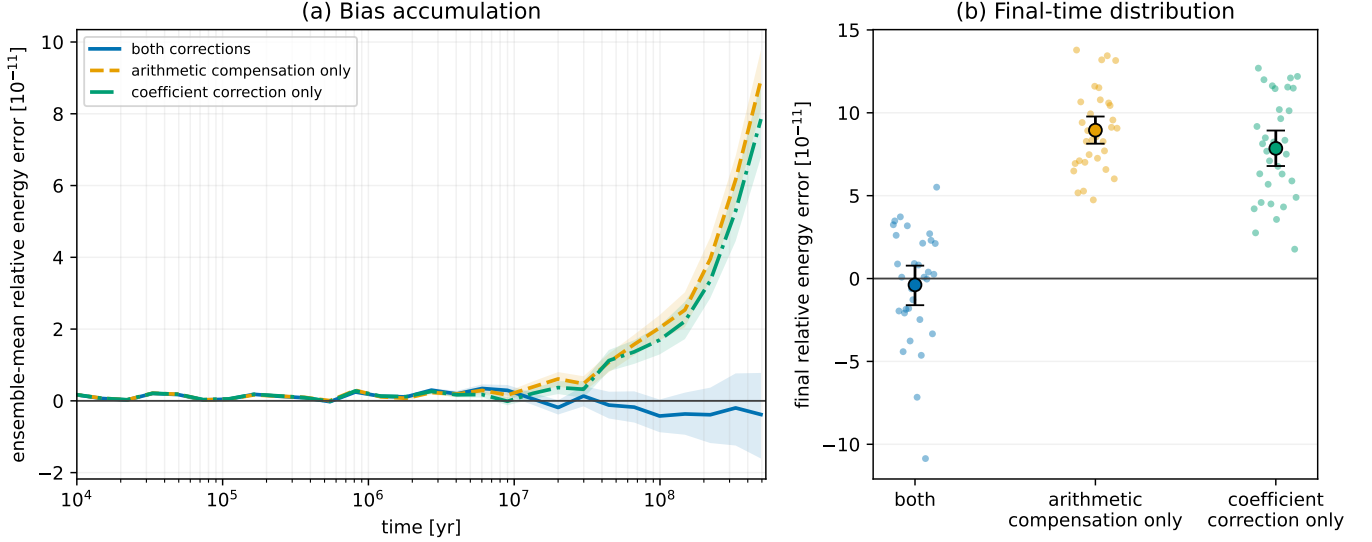


Figure 8. Roundoff-bias tests for sets of 32 Solar System integrations with both corrections, compensated summation only, and coefficient correction only. Left: ensemble-mean relative energy error as a function of time, shaded regions show two sigma confidence intervals. Right: final relative energy error for each simulation; small points show individual simulations, black points and error bars show the ensemble means and two sigma confidence intervals.

B. BRUTE-FORCE SEARCH FOR AN EFFICIENT KEPLER SOLVER

We tested different Kepler-solver sequences with one to three Newton or Halley refinement stages as well as Stiefel-function truncation orders of 7, 11, 15, and 19. Taken together, we tested 2336 different combinations. For each combination, we measured the wall time and the errors in e , a , longitude of pericenter, and mean anomaly for eight Kepler problems with eccentricities ranging from 0.30 to 0.72 and a 6 day timestep. We acknowledge that the fastest combination is ultimately depended on the specific planetary system benchmarked on. However, among the combinations that reached machine precision, several ones are almost equally fast. The well performing combinations typically start with one or two Halley steps to quickly reach a moderately accurate solution but then use Newton steps to achieve the final convergence to machine precision. Among the 2336 combinations, we select the fastest one. It uses a 9-term Halley step, an 11-term Halley step, and then 19-term Newton iterations with a convergence check and a fallback to the bisection method if the Newton method does not converge within 4 step. The accuracy over a broader parameter space has then been verified, see Figs. 2, 3, and 7.

C. ROUNDOFF-BIAS TESTS OF STIEFEL FUNCTIONS

We test the two roundoff corrections described in Section 2.4 with three versions of the final Stiefel-function evaluation where we include both corrections, compensated summation only, and coefficient correction only. Using the notation from Algorithm 2, the low-word recurrence is $D' = -zD + A(s_e - p_e) - C\delta_k$, where A enables compensation of arithmetic roundoff and C enables transport of the coefficient residual. We are thus testing $(A, C) = (1, 1)$, $(1, 0)$, and $(0, 1)$.

We integrate three sets of 32 simulations of the eight-planet Solar System with a 6-day timestep, general relativistic corrections, and an order-17 symplectic corrector for 3×10^{10} timesteps. The initial positions and velocities of each simulation are perturbed at the 10^{-15} level. Figure 8 shows the ensemble-mean energy error as a function of time for each set in the left panel and the final result for each simulation in the right panel.

When both corrections are used the ensemble-mean energy error is $(0.4 \pm 1.2) \cdot 10^{-11}$ at the final time, i.e. consistent with zero. With compensated summation only, the mean is $(8.6 \pm 0.8) \cdot 10^{-11}$; with coefficient correction only, it is $(7.9 \pm 1.1) \cdot 10^{-11}$. Thus only using either one of the two corrections produces a positive bias, whereas using both does not have any bias over this interval.

REFERENCES

- Abbot, D. S., Hernandez, D. M., Hadden, S., et al. 2023, The Astrophysical Journal, 944, 190
- Amdahl, G. M. 1967, in Proceedings of the April 18-20, 1967, Spring Joint Computer Conference, AFIPS '67 (Spring) (New York, NY, USA: Association for Computing Machinery), 483–485, doi: [10.1145/1465482.1465560](https://doi.org/10.1145/1465482.1465560)

- Applegate, J. H., Douglas, M. R., Gursel, Y., Sussman, G. J., & Wisdom, J. 1986, *Astronomical Journal*, 92, 176, doi: [10.1086/114149](https://doi.org/10.1086/114149)
- Batygin, K., & Laughlin, G. 2008, *The Astrophysical Journal*, 683, 1207
- Beust, H. 2003, *A&A*, 400, 1129, doi: [10.1051/0004-6361:20030065](https://doi.org/10.1051/0004-6361:20030065)
- Boué, G., Laskar, J., & Farago, F. 2012, *Astronomy and Astrophysics*, 548, A43, doi: [10.1051/0004-6361/201219991](https://doi.org/10.1051/0004-6361/201219991)
- Brouwer, D. 1937, *Astronomical Journal*, 46, 149, doi: [10.1086/105423](https://doi.org/10.1086/105423)
- Brown, G., & Rein, H. 2023, *MNRAS*, 521, 4349, doi: [10.1093/mnras/stad719](https://doi.org/10.1093/mnras/stad719)
- Cohen, C. J., & Hubbard, E. C. 1965, *AJ*, 70, 10, doi: [10.1086/109674](https://doi.org/10.1086/109674)
- Danowitz, A., Kelley, K., Mao, J., Stevenson, J. P., & Horowitz, M. 2012, *Communications of the ACM*, 55, 55
- Dehnen, W., & Hernandez, D. M. 2017, *MNRAS*, 465, 1201, doi: [10.1093/mnras/stw2758](https://doi.org/10.1093/mnras/stw2758)
- Dehnen, W., & Read, J. I. 2011, *The European Physical Journal Plus*, 126, 55, doi: [10.1140/epjp/i2011-11055-3](https://doi.org/10.1140/epjp/i2011-11055-3)
- Eckert, W. J., Brouwer, D., & Clemence, G. M. 1951, *Astronomical papers prepared for the use of the American ephemeris and nautical almanac ; v. 12*, 12, 1
- Esmailzadeh, H., Blem, E., St. Amant, R., Burger, D., & Barker, K. 2011, *IEEE Micro*, 32, 28
- Farrés, A., Laskar, J., Blanes, S., et al. 2013, *Celestial Mechanics and Dynamical Astronomy*, 116, 141, doi: [10.1007/s10569-013-9479-6](https://doi.org/10.1007/s10569-013-9479-6)
- Hernandez, D. M. 2016, *Monthly Notices of the Royal Astronomical Society*, 458, 4285, doi: [10.1093/mnras/stw569](https://doi.org/10.1093/mnras/stw569)
- Hernandez, D. M., & Bertschinger, E. 2015, *MNRAS*, 452, 1934, doi: [10.1093/mnras/stv1439](https://doi.org/10.1093/mnras/stv1439)
- Hernandez, D. M., & Dehnen, W. 2017, *Monthly Notices of the Royal Astronomical Society*, 468, 2614, doi: [10.1093/mnras/stx547](https://doi.org/10.1093/mnras/stx547)
- Hoang, N. H., Mogavero, F., & Laskar, J. 2022, *Monthly Notices of the Royal Astronomical Society*, 514, 1342, doi: [10.1093/mnras/stac1299](https://doi.org/10.1093/mnras/stac1299)
- Javaheri, P., Rein, H., & Tamayo, D. 2023, *The Open Journal of Astrophysics*, 6, 29, doi: [10.21105/astro.2307.05683](https://doi.org/10.21105/astro.2307.05683)
- Kinoshita, H., & Nakai, H. 1984, *Celestial Mechanics*, 34, 203, doi: [10.1007/BF01235802](https://doi.org/10.1007/BF01235802)
- Kinoshita, H., Yoshida, H., & Nakai, H. 1991, *Celestial Mechanics and Dynamical Astronomy*, 50, 59
- Laskar, J. 2008, *Icarus*, 196, 1, doi: [10.1016/j.icarus.2008.02.017](https://doi.org/10.1016/j.icarus.2008.02.017)
- Laskar, J., & Gastineau, M. 2009, *Nature*, 459, 817, doi: [10.1038/nature08096](https://doi.org/10.1038/nature08096)
- Laskar, J., & Robutel, P. 1995, *Celestial Mechanics and Dynamical Astronomy*, 62, 193, doi: [10.1007/BF00692088](https://doi.org/10.1007/BF00692088)
- Laskar, J., & Robutel, P. 2001, *Celestial Mechanics and Dynamical Astronomy*, 80, 39, doi: [10.1023/A:1012098603882](https://doi.org/10.1023/A:1012098603882)
- Mikkola, S. 1997, *Celestial Mechanics and Dynamical Astronomy*, 67, 145, doi: [10.1023/A:1008217427749](https://doi.org/10.1023/A:1008217427749)
- Moore, G. E. 1965, *Electronics*, 38, 114
- Moore, G. E. 1975, in *Technical Digest of the 1975 International Electron Devices Meeting (IEDM)*, Vol. 21 (IEEE), 11–13
- Newhall, X. X., Standish, E. M., & Williams, J. G. 1983, *A&A*, 125, 150
- Nitadori, K., Makino, J., & Hut, P. 2006, *NewA*, 12, 169, doi: [10.1016/j.newast.2006.07.007](https://doi.org/10.1016/j.newast.2006.07.007)
- Nobili, A., & Roxburgh, I. W. 1986, in *IAU Symposium, Vol. 114, Relativity in Celestial Mechanics and Astrometry. High Precision Dynamical Theories and Observational Verifications*, ed. J. Kovalevsky & V. A. Brumberg, 105–110
- Quinn, T. R., Tremaine, S., & Duncan, M. 1991, *AJ*, 101, 2287, doi: [10.1086/115850](https://doi.org/10.1086/115850)
- Rein, H., Dey, K., & Tamayo, D. 2026, arXiv e-prints, arXiv:2601.08019, doi: [10.48550/arXiv.2601.08019](https://doi.org/10.48550/arXiv.2601.08019)
- Rein, H., & Tamayo, D. 2015, *MNRAS*, 452, 376
- Rein, H., & Tamayo, D. 2019, *Research Notes of the AAS*, 3, 16, doi: [10.3847/2515-5172/aaff63](https://doi.org/10.3847/2515-5172/aaff63)
- Rein, H., Tamayo, D., & Brown, G. 2019, *Monthly Notices of the Royal Astronomical Society*, 489, 4632, doi: [10.1093/mnras/stz2503](https://doi.org/10.1093/mnras/stz2503)
- Saha, P., Stadel, J., & Tremaine, S. 1997, *Astronomical Journal*, 114, 409, doi: [10.1086/118485](https://doi.org/10.1086/118485)
- Stiefel, E. L., & Scheifele, G. 1975, *Linear and regular celestial mechanics. Perturbed two-body motion. Numerical methods. Canonical theory.*
- Tamayo, D., Cranmer, M., Hadden, S., et al. 2020, *Proceedings of the National Academy of Science*, 117, 18194, doi: [10.1073/pnas.2001258117](https://doi.org/10.1073/pnas.2001258117)
- Tanikawa, A., Yoshikawa, K., Okamoto, T., & Nitadori, K. 2012, *NewA*, 17, 82, doi: [10.1016/j.newast.2011.07.001](https://doi.org/10.1016/j.newast.2011.07.001)
- Wisdom, J. 2015, *AJ*, 150, 127, doi: [10.1088/0004-6256/150/4/127](https://doi.org/10.1088/0004-6256/150/4/127)
- Wisdom, J., & Holman, M. 1991, *Astronomical Journal*, 102, 1528, doi: [10.1086/115978](https://doi.org/10.1086/115978)
- Wisdom, J., Holman, M., & Touma, J. 1996, *Fields Institute Communications*, 10, 217
- Wisdom, J. L. 1981, PhD thesis, California Institute of Technology, Pasadena.